Evidence-Driven Memory Retrieval for a Multilingual, Provider-Agnostic AI Client

Two-page summary of the Turing AI evaluation

Ömer Girgin · October 2026 · Code, data and full results: github.com/girginomer10/turing-ai, folder **benchmarks/**. Code, analysis scripts and parts of the text were written with an AI coding assistant (Claude) that I directed and reviewed.

0.44 → 0.995	**22 · 9 · 6**	**+0.36**	**21–48%**
MRR@10 on personal memory, keyword search to the best pipeline (needs an embedding profile and a reranker; not the default)	embedding models (25 configurations) · rerankers · LLM judges	MRR for finding events again after the fading fix (95% CI +0.26 to +0.45)	of unanswerable queries still got memory injected by six LLM judges at the 0.75 bar (wide CIs)

Summary

Turing AI is a native iOS, iPadOS and macOS app (with an Android port) where people bring their own provider keys and keep one local, portable memory across 50 provider presets. Whether that memory helps depends on retrieval: finding the right earlier fact in Turkish or English among near-duplicates, and staying silent when nothing applies. I built two reproducible benchmarks around the app's real Swift retriever, evaluated 22 embedding models, 9 rerankers and 6 LLM judges with bootstrap confidence intervals, and used the results to find and fix five defects. Quality is MRR@10: the average of 1/rank of the first correct memory in the top 10 (1 = always first, 0 = never found).

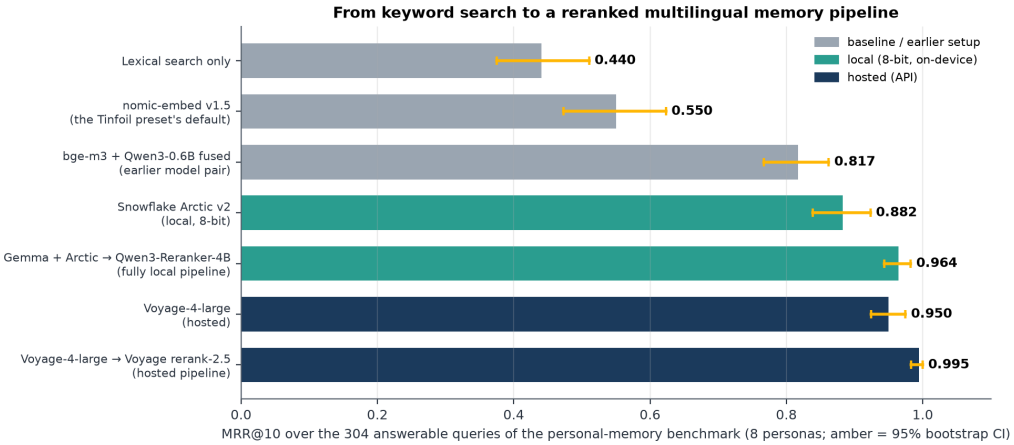


Figure 1. *MRR@10 on the personal-memory benchmark for seven systems, from lexical search to a hosted two-stage pipeline (95% bootstrap CIs). The fully local pipeline (0.964) is not significantly different from Voyage-4-large alone (0.950; paired difference +0.015, CI −0.013 to +0.042).*

How it was measured

- **Two datasets.** XQuAD (1,190 parallel Turkish/English questions) and a synthetic personal-memory set I built: 8 personas, 560 stored items, 360 queries including paraphrase, cross-lingual, hard-negative and 56 unanswerable ones.
- **Production code under test.** Everything runs the app's own retriever (*HybridMemoryRetriever*); Python re-implementations reproduce the Swift ranks in 42,840 of 42,840 cases.
- **Protocol.** Choices (fusion, fade policy, confidence bar) were made on DEV personas and reported on disjoint TEST personas; comparisons use paired bootstrap differences. Four blind annotator agents agreed with all 360 labels (κ = 1.0), which checks validity, not realism. The hosted study cost about $10.

Findings

- **Models differ far more on personal memory than on encyclopedic text** (0.55 to 0.95 against a 0.016 spread among strong models on XQuAD). Crossing languages separates them: nomic v1.5, Mistral embed and OpenAI text-embedding-3-small collapse (0.05, 0.19, 0.47) while Voyage-4-large keeps 0.925.
- **A reranker is the biggest second-stage win.** It lifts a fully local pipeline from 0.887 to 0.964 and a hosted one to 0.995, the benchmark's ceiling; a too-small reranker (Qwen3-0.6B) is worse than none. These numbers apply with Smart memory off. Fusing embedding models, by contrast, is not shown to help (+0.026, CI −0.003 to +0.060).
- **Staying silent is the hard part.** All six LLM judges keep 99–100% of correct answers but inject on 21–48% of unanswerable queries. A higher selection bar (0.85, chosen on DEV) cut injection from 41% to 23% on TEST at 0.999 → 0.994 answers kept; counts are small, so the direction is dependable and the size is not.

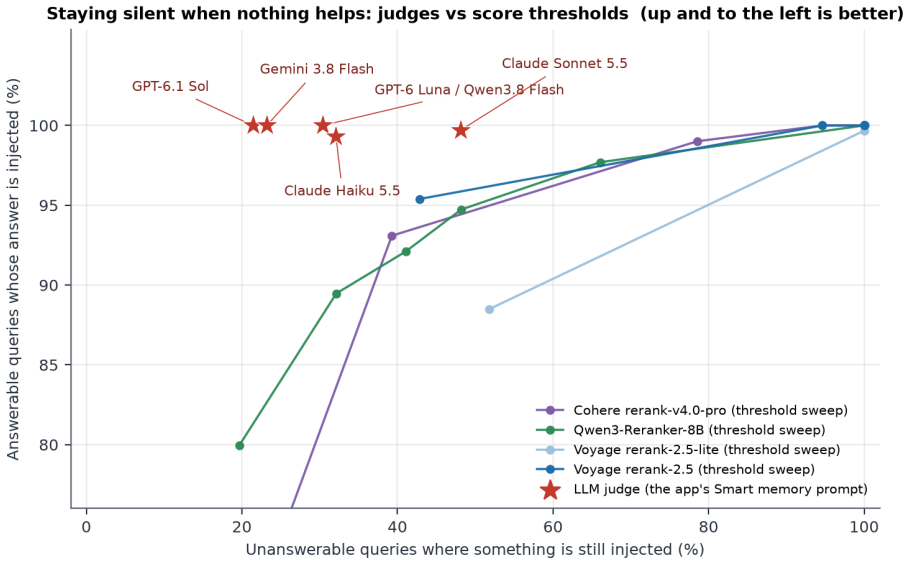


Figure 2. *Rerankers swept over thresholds (curves) against LLM judges (stars): dropping irrelevant injections costs correct answers. The comparison holds near 99% answer retention, not along the whole frontier.*

What the benchmarks changed in the product

Finding	Evidence	Change in the app
Semantic score was a flat rank percentile, so lexical noise decided the order	XQuAD 0.865 → 0.950; personal memory 0.572 → 0.733	Per-query min-max cosine in the blend
Event weight halved every 14 days; a month-old event was nearly unfindable	Answer-is-event MRR 0.48 → 0.84	45-day half-life, 0.4 floor, shield for top-ranked events
Models need documented query/document prefixes; app sent raw text	+0.073 EmbeddingGemma, +0.065 Arctic, +0.069 Nemotron-8B	Correct formats per model; stored vectors rebuilt
Smart memory judged only the first 40 lexical matches, holding the answer for 81% of queries	Embedding top 20 holds it for 100%	Judge sees the embedding-ranked top 20
Strict JSON decoder discarded correct answers	Claude Sonnet 5.5 valid answers 78.6% → 98.3%	Tolerant decoder, still rejects invented IDs
Judges kept every entry rated ≥ 0.75	Injection on unanswerable 41% → 23% (TEST, six judges)	Selection bar raised to 0.85

What a default user gets. *No embedding profile exists until the user adds one, so out of the box the app ranks lexically and Smart memory judges the first 40 matches. The 0.964 and 0.995 pipelines need an embedding profile, a reranker and Smart memory switched off.*